ALACRITY / FLUX · مقالهٔ پژوهشی · ۱۰ مهر ۱۴۰۵

v6: مدل کوچک حساب می‌کند، نه حدس

آزمایش واقعی موتور استدلال برنامه‌محور، آبشار Luna به Sol، و مقایسه با حالت عادی Flux

گزارش تحقیق و توسعه · ۲ اکتبر ۲۰۲۶ · ۴۴۸ فراخوانی واقعی gpt-6-luna و gpt-6.1-sol · همهٔ اعداد از فایل‌های نتیجه بازسازی شده‌اند

چکیده. Luna در «حالت عادی» مسئلهٔ استدلالی را مستقیم و بدون محاسبه جواب می‌دهد. در این آزمایش ۵۶ مسئلهٔ محاسبه‌پذیر، هر کدام دو بار، اجرا شد: ۴۰ مسئلهٔ تازه در ده خانواده (ده مسئله به فارسی) و ۱۶ مسئلهٔ آزمایش‌های قبلی. Luna در حالت عادی ۲۹٫۵٪ پاسخ درست داد. در v6، Luna به‌جای جواب دو برنامه با دو روش مستقل می‌نویسد، برنامه‌ها اجرا و مقایسه می‌شوند و فقط در صورت عدم توافق، Sol وارد می‌شود. نتیجه ۹۹٫۱٪ بود (۱۱۱ از ۱۱۲)؛ یعنی **۹۸٫۷٪ خطای کمتر** از حالت عادی، در حالی که Sol فقط در ۲۶٫۸٪ درخواست‌ها فراخوانی شد. Sol مستقیم ۱۰۰٫۰٪ درست بود. زمان میانهٔ v6 ۱۰٫۴ ثانیه بود، در برابر ۱۲٫۳ ثانیه برای Sol و ۷٫۲ ثانیه برای Luna عادی. در بار کاری ترکیبی (۵۰٪ واقعیت، ۳۰٪ استدلال، ۲۰٪ کد)، v6 به دقت ۹۹٫۷٪ در برابر ۷۶٫۰٪ حالت عادی می‌رسد و میانگین زمان ۱۵٪ کمتر می‌شود. محدودیت‌ها: یک توافق غلط باقی ماند، میانگین زمان استدلال از Sol سریع‌تر نیست، و «۹۵٪ سریع‌تر» فقط در لایهٔ واقعیت محقق شد.

دقت استدلال: عادی → v6	کاهش خطا نسبت به حالت عادی	سهم Sol در v6	توافق غلط
۹۹٫۱٪ → ۲۹٫۵٪	۹۸٫۷٪	۲۶٫۸٪	۱٫۲٪
۱۱۲ پاسخ	هدف: ≥ ۹۵٪	هدف: ≥ ۳۵٪	۱ از ۸۲ توافق

۱. چرا Luna در استدلال شکست می‌خورد؟
۲. معماری v6
۳. چگونه آزمایش کردیم
۴. دقت: از حدس تا محاسبه
۵. توافق دو برنامه و تنها خطای باقی‌مانده
۶. به تفکیک خانواده و زبان
۷. سرعت
۸. در حالت عادی چقدر بهتر می‌شود؟
۹. آیا به ۹۵٪ رسیدیم؟
۱۰. محدودیت‌ها و قدم بعدی
۱۱. منابع و بازتولید

۱. چرا Luna در استدلال شکست می‌خورد؟

در سه آزمایش قبلی (v3، v5 و پایلوت مسیریاب) Luna در کد و سؤال‌های حافظه‌ای خوب بود، اما در استدلال بسته فقط ۳۳ تا ۴۱ درصد درست جواب داد. نگاه به خروجی‌ها علت را نشان داد: میانگین خروجی Luna برای این سؤال‌ها **۱۴ توکن** بود؛ یعنی فقط یک عدد، بدون هیچ محاسبه‌ای. مسئله‌هایی که در آن‌ها شکست خورد (CRT، امید ریاضی الگوی تاس، احتمال شرطی، کوتاه‌ترین مسیر) دقیقاً از نوعی‌اند که یک برنامهٔ کوتاه بی‌خطا حل می‌کند. در همان آزمایش‌ها، Luna در نوشتن کد بین ۸۶ تا ۱۰۰ درصد موفق بود.

فرضیهٔ v6: اگر از Luna به‌جای جواب، **برنامه‌ای که جواب را حساب می‌کند** بخواهیم و دو برنامهٔ مستقل یکدیگر را کنترل کنند، بخش بزرگی از فاصله با Sol بسته می‌شود؛ و Sol فقط وقتی لازم است که این کنترل شکست بخورد. این ایده از PAL و Program-of-Thought الهام گرفته، ولی آن پژوهش‌ها بازتولید نشده‌اند.

۲. معماری v6

v6 هر درخواست را در ارزان‌ترین لایه‌ای پاسخ می‌دهد که بتواند درستی را نشان دهد. تشخیص نوع درخواست قطعی است (بدون مدل) و کمتر از یک میلی‌ثانیه طول می‌کشد.

۰. کش	۱. پاسخ مستقیم	۲. موتور استدلال	۳. کد	۴. استدلال باز
درخواست تکراری	واقعیت اثبات‌شده از حافظه	دو برنامهٔ Luna، اجرای دقیق، توافق	Luna + تست عمومی	Sol مستقیم
۰ فراخوانی	۰ فراخوانی · ۰٫۶۴ ms	۲ فراخوانی موازی	Sol در صورت شکست	بدون checker؛ unverified

موتور استدلال در چهار قدم

صورت‌بندی: Luna دو برنامهٔ `solve()` می‌نویسد، همزمان و با دو راهنمای روش: A تحلیلی (فرمول یا الگوریتم استاندارد) و B ساختی (شمارش مستقیم، برنامه‌نویسی پویا یا حل دقیق معادلات حالت). خروجی فقط کد است، بدون توضیح. **اجرا:** worker محدود با حساب دقیق (`int` و `Fraction`)، بدون فایل و شبکه و eval، با سقف زمان. میانهٔ اجرا ۴۴ میلی‌ثانیه بود. **بررسی:** دو جواب پس از نرمال‌سازی با قالب خواسته‌شده (عدد صحیح، احتمال، تعداد رقم اعشار) برابر باشند و از بررسی سلامت (بازه، نوع) عبور کنند. **آبشار:** در عدم توافق، خطای اجرا یا شکست بررسی سلامت، Sol مستقل و بدون دیدن برنامه‌ها حل می‌کند.

۳. چگونه آزمایش کردیم

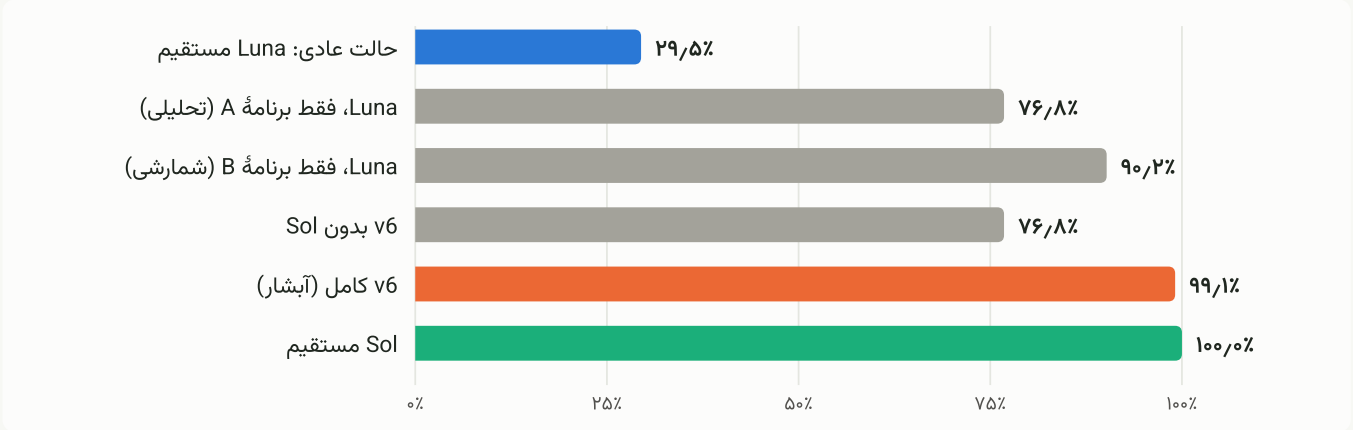
۵۶ مسئله: ۴۰ مسئلهٔ تازه از ده خانواده که با seed ثابت ساخته شدند و جواب مرجعشان با کد مستقل محاسبه شد (هر خانواده چهار مسئله، یکی به فارسی)، و ۱۶ مسئلهٔ R00 تا R15 از آزمایش‌های قبلی. هر مسئله دو بار اجرا شد و در هر نمونه چهار فراخوانی مستقل داشت: Luna مستقیم (حالت عادی)، برنامهٔ A، برنامهٔ B و Sol مستقیم. جمعاً ۴۴۸ فراخوانی از مسیر Codex CLI با reasoning کم، شش فراخوانی همزمان. کد، promptها، جواب‌های مرجع و آستانهٔ hedge (۱۰ ثانیه) پیش از اولین فراخوانی با hash منجمد شدند. همهٔ مسیرها با یک قاعدهٔ نمره‌دهی ارزیابی شدند. v6 از همین پاسخ‌ها ساخته می‌شود: اگر A و B توافق کنند جواب v6 همان است؛ وگرنه جواب Sol. زمان v6 از زمان‌های اندازه‌گیری‌شده بازپخش می‌شود (بیشینهٔ A و B، به‌اضافهٔ Sol در صورت ارجاع).

> این آزمون تأثیر Sol را در مسیر ارجاع با پاسخ مستقیم Sol می‌سنجد، نه با یک prompt جداگانهٔ escalation. زمان‌ها زیر بار شش فراخوانی همزمان اندازه‌گیری شده‌اند. تشخیص نوع سؤال روی همین مجموعه ۹۴٫۶٪ offline دقیق بود و روی تسک‌های زندهٔ این آزمون اثری نداشت (همه به موتور استدلال رسیدند).

۴. دقت: از حدس تا محاسبه

تنها با تغییر قالب خروجی از «جواب» به «برنامه»، دقت Luna از ۲۹٫۵٪ به ۷۶٫۸٪ (برنامهٔ تحلیلی) و ۹۰٫۲٪ (برنامهٔ ساختی) رسید. ترکیب دو برنامه با ارجاع به Sol، دقت را به ۹۹٫۱٪ رساند.

نمودار ۱. درصد پاسخ درست نهایی در هر مسیر



۱۱۲ پاسخ (۵۶ مسئله × ۲ نمونه). میله‌های خاکستری اجزای میانی v6 هستند و مسیر مستقل محصول نیستند. منبع: R1.

اختلاف v6 با حالت عادی، با bootstrap جفتی روی مسئله‌ها، بین ۵۸٫۰ و ۸۱٫۲ واحد درصد به سود v6 است. فاصله با Sol مستقیم بین صفر و ۲٫۷ واحد درصد به زیان v6 است؛ تنها خطای v6 همان توافق غلط بخش بعد است.

۵. توافق دو برنامه و تنها خطای باقی‌مانده

در ۷۳٫۲٪ درخواست‌ها دو برنامه توافق کردند و پاسخ بدون Sol داده شد. دقت پاسخ‌های توافقی ۹۸٫۸٪ بود. در باقی موارد، Sol پاسخ داد و همهٔ آن‌ها درست بودند.

نمودار ۲. سرنوشت ۱۱۲ درخواست در موتور استدلال



باریکهٔ قرمز یک درخواست است؛ مقدار هر بخش با نگه‌داشتن نشانگر هم دیده می‌شود. منبع: R1.

توافق غلط: وقتی هر دو برنامه یک‌جور اشتباه می‌کنند

سؤال: «Using unlimited coins of values 2, 4, 7, 17, what is the minimum number of coins needed to make exactly 83?». هر دو برنامهٔ Luna سکهٔ ۴ را نادیده گرفتند و هر دو به ۹ رسیدند؛ جواب درست ۷ است (۱۷+۱۷+۱۷+۷+۴+۴+۴). این همان خطر «بدفهمی مشترک» است که در سند معماری پیش‌بینی شده بود. روش دوم قرار بود مستقل باشد، اما هر دو برنامه از یک برداشت غلط از صورت مسئله شروع کردند.

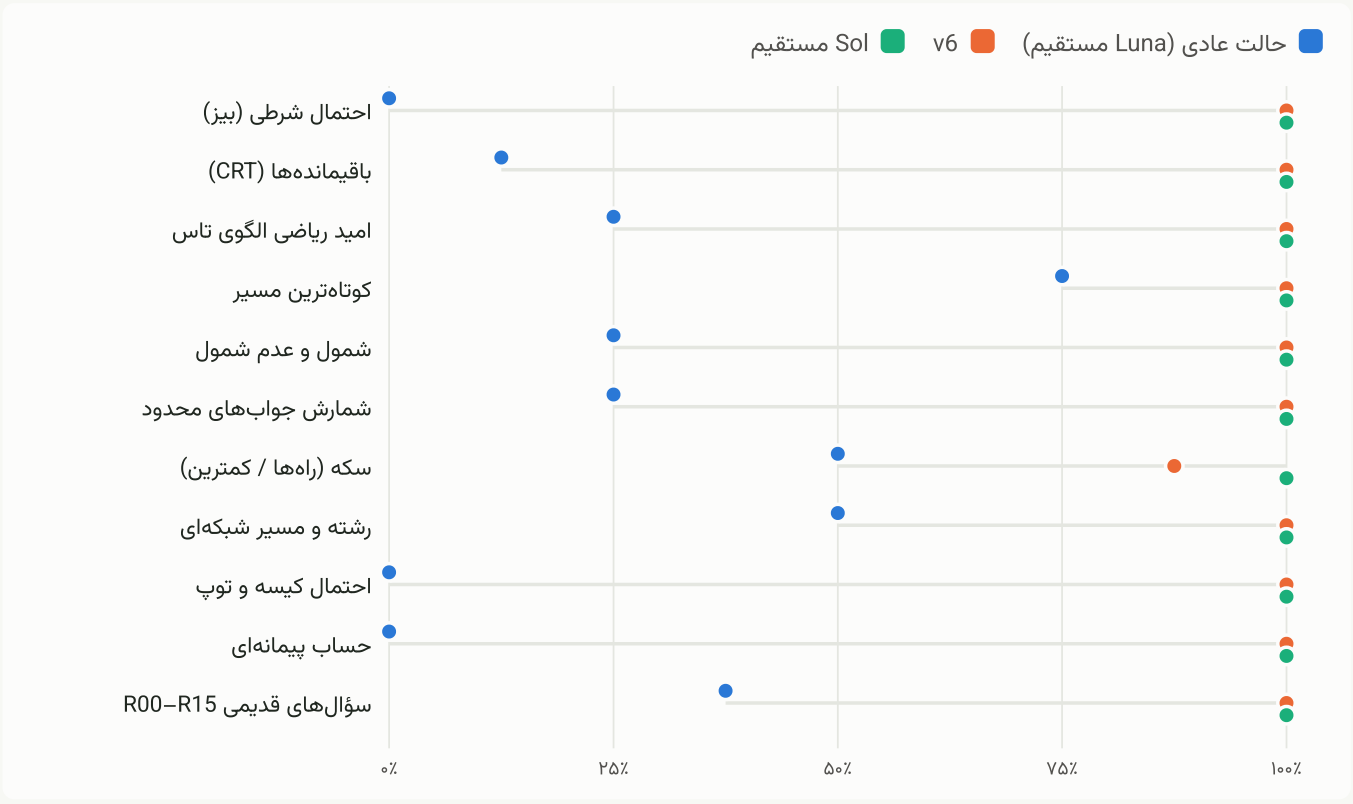
```
def solve():
    best = 83
    for n17 in range(83 // 17 + 1):
        for n7 in range((83 - 17 * n17) // 7 + 1):
            rem = 83 - 17 * n17 - 7 * n7
            if rem % 2 == 0:
                best = min(best, n17 + n7 + rem // 2)
    return best
```

پنج برنامه از ۲۲۴ برنامه اجرا نشدند: دو خطای زمان اجرا (`min()` و `max()` روی دنبالهٔ خالی)، دو مورد رد `__import__` توسط sandbox، و یک `import *` داخل تابع. همه طبق طراحی به Sol ارجاع شدند.

۶. به تفکیک خانواده و زبان

در پنج خانواده (بیز، شمول و عدم شمول، حساب پیمانه‌ای، کوتاه‌ترین مسیر و تا حدی رشته‌ها) دو برنامه تقریباً همیشه توافق کردند و Sol لازم نشد. در «امید ریاضی الگوی تاس» هیچ توافقی رخ نداد و همه به Sol رفتند؛ Luna این مسئله را نه مستقیم و نه با برنامه حل کرد، ولی آبشار جواب درست را تضمین کرد.

نمودار ۳. دقت به تفکیک خانواده: حالت عادی، v6 و Sol



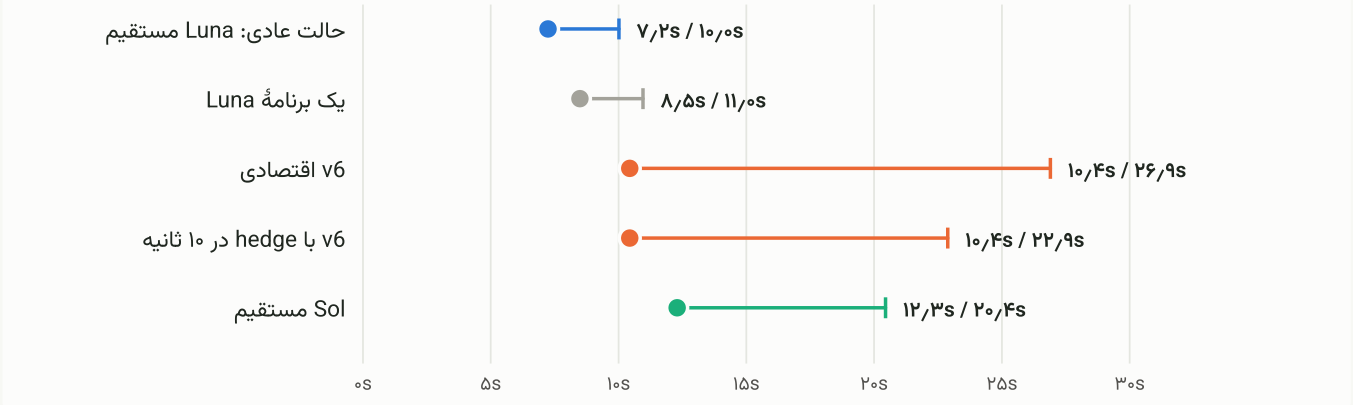
هر ردیف ۸ پاسخ (۴ مسئله × ۲ نمونه) و ردیف آخر ۳۲ پاسخ. نقطه‌ها روی هم افتاده‌اند وقتی v6 و Sol هر دو ۱۰۰٪ هستند. منبع: R1.

در ده مسئلهٔ فارسی، دقت حالت عادی ۲۰٫۰٪ بود و دقت v6 و Sol هر دو ۱۰۰٫۰٪. در ۱۶ مسئلهٔ قدیمی، دقت حالت عادی ۳۷٫۵٪ و دقت v6 برابر ۱۰۰٫۰٪ بود.

۷. سرعت

v6 در استدلال **از Luna عادی کندتر** است، چون به‌جای یک عدد، دو برنامه تولید می‌شود؛ ولی میانهٔ زمانش از Sol مستقیم کمتر است. دم توزیع (p95) بدتر از Sol است، چون در ارجاع، Sol بعد از تمام‌شدن دو برنامه شروع می‌شود.

نمودار ۴. زمان پاسخ استدلال: نقطه میانه و خط تا p95



زمان کامل هر فراخوانی Codex CLI، شامل حدود ۵ ثانیه سربار ثابت. اجرای برنامه‌ها در worker با میانهٔ ۴۴ ms به آن اضافه شده است. منبع: R1.

آستانهٔ hedge (۱۰ ثانیه) پیش از آزمایش منجمد شد و کم بود: چون یک برنامهٔ Luna به‌تنهایی میانهٔ ۸٫۵ ثانیه دارد، hedge در ۵۶٫۲٪ درخواست‌ها Sol را

فعال کرد و فقط p95 را از ۲۶٫۹ ثانیه به ۲۲٫۹ ثانیه رساند. این یک مشاهدهٔ پس از آزمایش است و آستانهٔ بهتر باید روی دادهٔ تازه انتخاب شود.

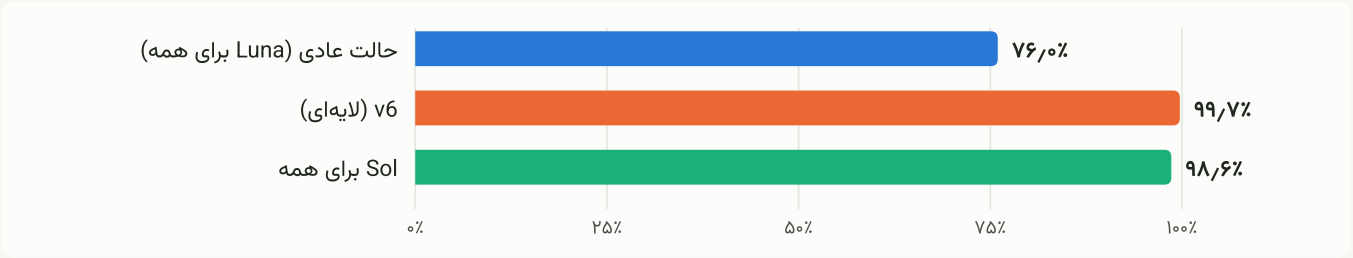
بزرگ‌ترین بخش زمان، سربار انتقال است: یک پاسخ هفت‌توکنی از Luna حدود ۵ ثانیه طول می‌کشد. تا وقتی هر فراخوانی یک پردازهٔ تازهٔ CLI است، استدلال نمی‌تواند به ۹۵٪ سرعت برسد.

۸. در حالت عادی چقدر بهتر می‌شود؟

«حالت عادی» یعنی همان کاری که Flux امروز می‌کند: Luna به همهٔ درخواست‌ها جواب می‌دهد (برای واقعیت‌ها با بستهٔ حافظهٔ v5). برای مقایسهٔ منصفانه، سه نوع درخواست با سهم پایلوت قبلی ترکیب شدند: ۵۰٪ سؤال واقعیتی، ۳۰٪ استدلال، ۲۰٪ کد.

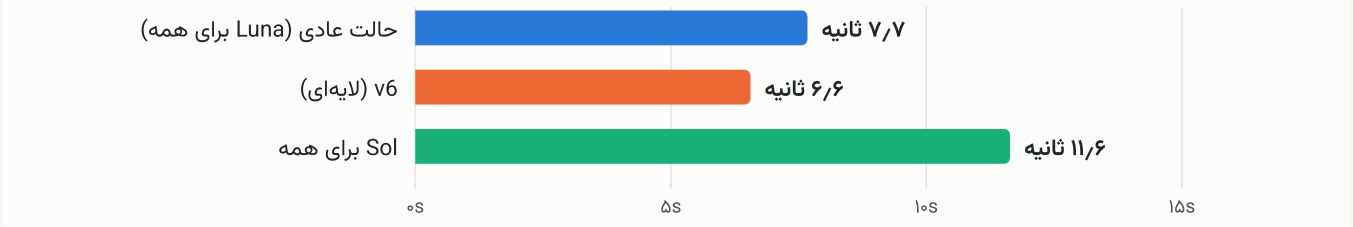
- **استدلال:** همین آزمایش (R1).
- **واقعیت:** حالت عادی از اجرای واقعی Luna+v5 روی ۱۰ سؤال (R4)؛ v6 از پاسخ مستقیم offline روی همان ۱۰ سؤال (R3)؛ Sol از ۳۶ پاسخ دانشی پایلوت مسیریاب (R5).
- **کد:** Luna و Sol از پایلوت مسیریاب (R5، بدون C04). سطر v6 **تخمین** است: Luna، و در صورت شکست تست عمومی، Sol؛ با این فرض که تست عمومی شکست را می‌گیرد.

نمودار ۵. دقت در بار کاری ترکیبی



ترکیب وزنی اجزای اندازه‌گیری‌شده؛ کد در v6 تخمین است. منبع: R1، R3، R4، R5.

نمودار ۶. میانگین زمان هر درخواست در بار کاری ترکیبی



سؤال واقعیتی در v6 زیر یک میلی‌ثانیه است؛ بنابراین میانگین v6 از حالت عادی کمتر است، با وجود اینکه استدلال در v6 کندتر است.

سنجه در بار ترکیبی	حالت عادی	v6	Sol برای همه
دقت	۷۶٫۰٪	**۹۹٫۷٪**	۹۸٫۶٪
میانگین زمان	۷٫۶۷ ثانیه	**۶٫۵۶ ثانیه**	۱۱٫۶۴ ثانیه
فراخوانی مدل در هر درخواست	۱٫۱۰	**۰٫۹۱**	۱٫۰۰
فراخوانی Sol در هر درخواست	۰	**۰٫۱۱**	۱

نسبت به حالت عادی: خطا ۹۸٫۹٪ کمتر، میانگین زمان ۱۴٫۵٪ کمتر، و تعداد فراخوانی مدل ۱۷٫۴٪ کمتر. نسبت به «Sol برای همه»: دقت مشابه، میانگین زمان ۴۳٫۶٪ کمتر و فراخوانی ۸۹٫۱٪ Sol کمتر.

نمونهٔ پاسخ‌های واقعی (نمونهٔ اول هر مسئله)

مسئله	حالت عادی	برنامهٔ A / B	v6	مرجع
احتمال شرطی (بیز) bayes_3	✗ 0.359425	0.360157 / 0.360157	✓ 0.360157 توافق دو برنامه	0.360157
باقیمانده‌ها (CRT) crt_2	✗ 5211	5901 / 5901	✓ 5901 توافق دو برنامه	5901
امید ریاضی الگوی تاس pattern_1	✗ 258	252 / 222	✓ 222 ارجاع به Sol	222
احتمال کیسه و توپ urn_1	✗ 0.688887	0.677245 / –	✓ 0.677245 ارجاع به Sol	0.677245
سکه (راه‌ها / کمترین) coins_1	✓ 7	9 / 9	✗ 9 توافق دو برنامه	7
سؤال‌های قدیمی R00–R15 R03	✗ 583	79 / 79	✓ 79 توافق دو برنامه	79
سؤال‌های قدیمی R00–R15 R04	✗ 258	215 / 258	✓ 216 ارجاع به Sol	216
حساب پیمانه‌ای modular_1	✗ 622	727 / 727	✓ 727 توافق دو برنامه	727

۹. آیا به ۹۵٪ رسیدیم؟

معیارها پیش از اجرا در سند معماری (R6) تعریف شده بودند:

معیار	هدف	نتیجه	وضعیت
کاهش خطای استدلال نسبت به حالت عادی	≤ ۹۵٪	۹۸٫۷٪	برآورده شد
فاصلهٔ کیفیت با Sol مستقیم (حد پایین بازهٔ ۹۵٪)	افت ≤ ۲ واحد	۲٫۷ واحد افت	برآورده نشد
نرخ توافق غلط	≤ ۲٪	۱٫۲٪	برآورده شد
سهم Sol در استدلال	≥ ۳۵٪	۲۶٫۸٪	برآورده شد
زمان میانهٔ استدلال ≥ Sol مستقیم	≥ ۱۲٫۳s	۱۰٫۴s	برآورده شد
لایهٔ واقعیت: ≥ ۹۵٪ سریع‌تر از حالت عادی	≤ ۹۵٪	۹۹٫۹۹٪	برآورده شد
دقت تشخیص نوع سؤال (offline)	≤ ۹۵٪	۹۴٫۶٪	برآورده نشد

قوی‌تر: بله. خطای استدلال ۹۸٫۷٪ کمتر از حالت عادی شد. اما حد پایین بازهٔ اطمینان فاصله با Sol کمی از مرز ۲ واحد درصد عبور کرد و یک توافق غلط باقی ماند؛ با ۱۱۲ پاسخ، برابری با Sol ثابت نمی‌شود. **سریع‌تر:** فقط در لایهٔ واقعیت (حدود ۱۰۰۰۰ برابر سریع‌تر). در استدلال، v6 در میانه از Sol سریع‌تر است ولی از Luna عادی کندتر؛ در بار ترکیبی میانگین زمان ۱۴٫۵٪ کم شد، نه ۹۵٪.

۱۰. محدودیت‌ها و قدم بعدی

- مسئله‌ها همه محاسبه‌پذیر بودند؛ استدلال باز (توضیح، داوری) آزموده نشد.
- هر مسئله دو نمونه داشت؛ ۵۶ مسئله برای ادعای عمومی کم است.
- سطر کد در مقایسهٔ ترکیبی تخمین است و repair آن اجرا نشد.
- هزینهٔ دلاری در مسیر اشتراک گزارش نمی‌شود (`billing_usd=null`).
- sandbox، worker سیستم‌عامل نیست؛ برای محصول باید ایزوله شود.

قدم بعدی: (۱) کاهش توافق غلط با یک قدم «بازنویسی صورت مسئله» یا روش سوم از Sol فقط برای خانواده‌های پرخطر؛ (۲) آستانهٔ hedge بر اساس p90 یک برنامه؛ (۳) نشست ماندگار یا API برای حذف ۵ ثانیه سربار هر فراخوانی؛ (۴) اجرای مسئله‌های باز و کد با repair واقعی.

۱۱. منابع و بازتولید

شناسه	فایل	SHA-256 کوتاه
R1	`experiments/request_memory_v6_20261002/results.json`	`adc3529f81285c90`
R2	`experiments/request_memory_v6_20261002/frozen-plan.json`	`4d55845a42a9e802`
R3	`experiments/request_memory_v6_20261002/fact-tier-results.json`	`3991a245ee4c68a5`
R4	`experiments/luna_v4_v5_live_20261002/results.json`	`cc2646dfbf0c11ce`
R5	`experiments/router_pilot_20261002/results.json`	`785eeaf52f935a6d`
R6	`experiments/request_memory_v6_design_20261002/ARCHITECTURE.fa.md`	`3b08689ba7c04e1c`

بازتولید: `cd experiments/request_memory_v6_20261002`، سپس `python -m pytest test_v6.py` (۱۲ تست offline)، `python run_v6.py analyze` (بدون فراخوانی مدل؛ از پاسخ‌های ذخیره‌شده) و `python docs/reasoning-v6-20261002/build_article_v6.py`. اجرای دوبارهٔ `run_v6.py live` پاسخ‌های ذخیره‌شده را بازاستفاده می‌کند. پژوهش‌های الهام‌بخش: PAL (arXiv:2211.10435)، Program of Thoughts (arXiv:2211.12588)، FrugalGPT (arXiv:2305.05176) و The Tail at Scale.

پایان مقاله · v6 هنوز به Flux اصلی متصل نشده است · برای ساخت این سند هیچ فراخوانی تازهٔ مدل انجام نشد.